

## Refresher: Statistics and Matrix Algebra

Yongjin Park

07 January 2026

## Slide credits

---

- ▶ Keegan Korthauer

These are wonderful online books (hyperlinks):

- ▶ Modern Statistics for Modern Biology
- ▶ Introduction to Data Science
- ▶ Biomedical Data Science

## Two challenges in genomic data analysis

---

- ▶ **Uncertainty**/noise distributions due to the nature of data-generating process
  - ▶ **High-dimensional** and big data because of many variables and many samples
- 
- ▶ The last lecture was to introduce genomics to statisticians

## Today's goal is to introduce statistics to biologists

---

Basic tools/frameworks for:

1. How can we deal with the **uncertainty** of genomic data (statistics)?
2. How can we effectively represent **high dimensional data** (matrix algebra)?

*Note:* In one lecture, it is impossible to teach you everything.

## Today's lecture

---

### Statistics Refresher

### Matrix Algebra Primer

### Matrix Algebra in Statistics

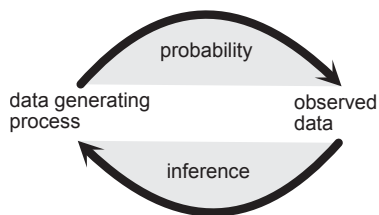
## What is Statistics?

---

- ▶ The field of statistics concerns the science of **collecting**, **analyzing/modeling**, **interpreting** data and **communicating uncertainty** about the results
- ▶ Statistics is **not** a collection of generic “recipes” to follow
- ▶ Our (statistical) goals for this course:
  - ▶ gain a rigorous understanding of core principles of common analyses of high-dimensional biological data
  - ▶ build solid foundation to follow up on specific topics

## Probability and Inference

---



- ▶ Probability will give insights into modelling.
  - ▶ Given a data generating process, we can we expect in data?
- ▶ We will estimate (recover) unknown from data by inference.
  - ▶ Given the data, what can we know about a plausible model?

Wasserman, *All of Statistics* (2003)

$$X \sim P(\theta)$$

vs.

$$\hat{\theta} \leftarrow \arg \max \log P(\text{Data}|\theta)$$

## A real example: mutation in HIV genome

---

- ▶ Human Immunodeficiency Virus (HIV) Genome:  $10^4$  nucleotides
- ▶ Mutation rate:  $5 \times 10^{-4}$  per nucleotide
- ▶ How many mutations are expected to happen in each round of genome (RNA) replication?

See MSMB Chapter 1.

## Expectation

- Suppose that a mutation can happen at any position with an equal probability.
- How many mutations are **expected** over all the genome per each replication?

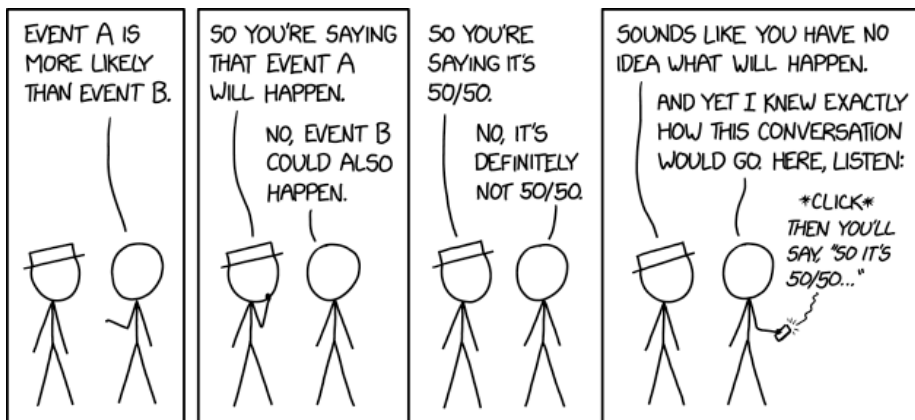
See MSMB Chapter 1.

## A Bernoulli trial

We can have two possible outcomes (a variable) for each genomic location  $i \in [10^4]$ :

$$X_i = \begin{cases} 1 & \text{mutation occurred} & \text{with probability } 5 \times 10^{-4} \\ 0 & \text{no mutation} & \text{with probability } 1 - 5 \times 10^{-4} \end{cases}$$

## So, will you see mutation or not?



XKCD on "prediction"

## Remark on the mutation indicator variable $X$

- It's like a coin flipping with an unequal chance of head and tail.
- We call a variable like  $X$ : a **Random Variable**.
- We set a Bernoulli random variable  $X = 1$ , assigning the value 1 for an outcome of the mutation event (head), or  $X = 0$  for the non-mutation (tail).
- It is a "random" variable, not a deterministic variable, because we assign the value with some probability.

## Probability of a Bernoulli trial

More succinctly, we can write the probability of the event:

$$P\{X_i = 1\} = 5 \times 10^{-4}$$

for all  $i \in [10^4]$ .

- More formally, probability maps an event to a long-run **frequency**.
- In order for a function to be a probability distribution/measure, it needs to satisfy the probability axioms.

## Each Bernoulli trial at each nucleotide

We have mutation rate

$$p = 5 \times 10^{-4},$$

with

$$n = 1$$

trial.

$$f(x; n, p) =$$

```
size <- 1
prob <- 5 * 10^-4
```

Okay, let's use R to play with this Bernoulli distribution. Can you find any function related to Bernoulli?

## Bernoulli distribution

### Bernoulli distribution

We write

$$X \sim \text{Bernoulli}(p)$$

for a random variable  $X$  following Bernoulli distribution with the rate parameter  $p$ .

- What is  $\sum_{x \in \{0,1\}} f(x)$ ?
- Can we have  $x \notin \{0,1\}$ ?

### Bernoulli probability function

Then the corresponding probability function is

$$f(x) = p^x(1-p)^{1-x}$$

where  $x \in \{0,1\}$ .

The lower case  $x$  is a particular value of the random variable  $X$ .

## Bernoulli is a special case of Binomial distribution

### Binomial distribution

We write

$$X \sim \text{Binomial}(n, p)$$

for a random variable  $X$  counts the number of "success" events over total  $n$  trials with the success rate  $p$ .

### Binomial probability function

Then the corresponding probability function is

$$f(x) = \binom{n}{x} p^x (1-p)^{n-x}$$

where

$$\binom{n}{x} = \frac{n!}{x!(n-x)!}$$

and

$$x \in \{0, 1, \dots, n\}.$$

## Each Bernoulli trial at each nucleotide

We have mutation rate

$$p = 5 \times 10^{-4},$$

with

$$n = 1$$

trial.

$$f(x; n, p) =$$

```
size <- 1
prob <- 5 * 10^-4

## repeat experiments 10 times
rbinom(10, size = size, prob = prob)
## [1] 0 0 0 0 0 0 0 0 0 0
```

Are we seeing any mutation event?

## Let's repeat Bernoulli trials many times

- In the HIV genome, what if we have  $n = 100$  nucleotides?
- We still assume that each nucleotide can have mutation with  $p = 5 \times 10^{-4}$ .

```
size <- 100
prob <- 5 * 10^-4

## repeat experiments 10 times
rbinom(10, size = size, prob = prob)
## [1] 0 0 0 0 0 0 0 1 0 0
```

How about now?

## Let's repeat Bernoulli trials many times

- What if we have  $n = 10^4$  nucleotides?
- We still assume that each nucleotide can have mutation with  $p = 5 \times 10^{-4}$ .

```
size <- 10^4
prob <- 5 * 10^-4

## repeat experiments 10 times
rbinom(10, size = size, prob = prob)
## [1] 3 10 8 4 3 7 4 3 3 2
```

How about now?

## Does rbinom generate the same value?

```
size <- 10^4
prob <- 5 * 10^-4

## repeat experiments 10 times
set.seed(1)
rbinom(10, size, prob)
## [1] 4 4 5 8 3 8 9 6 6 2
```

```
size <- 10^4
prob <- 5 * 10^-4

## repeat experiments 10 times
set.seed(2)
rbinom(10, size, prob)
## [1] 3 6 5 3 9 9 3 7 5 5
```

## What R can do for us

If you type `?rbinom` or `help(rbinom)` for `Binomial(size,prob)`:

The Binomial Distribution

Description:

Density, distribution function, quantile function and random generation for the binomial distribution with parameters 'size' and 'prob'.

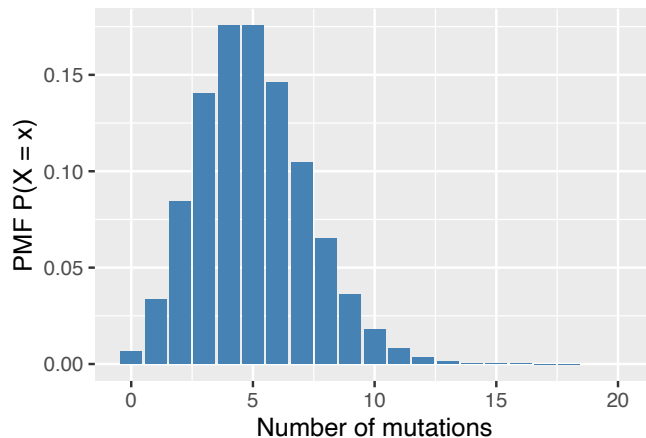
This is conventionally interpreted as the number of 'successes' in 'size' trials.

Usage:

```
dbinom(x, size, prob, log = FALSE)
pbinom(q, size, prob, lower.tail = TRUE, log.p = FALSE)
qbinom(p, size, prob, lower.tail = TRUE, log.p = FALSE)
rbinom(n, size, prob)
```

## Probability function dbinom

Binomial(n=10000, p=5e-04)



$f(x) = P(X = x)$  for each value of  $x$

## Functions related to probability

Probability Density Function (PDF)

► Discrete  $x \in \mathcal{X}$ :

$$f(x) = P(X = x)$$

► Continuous  $x \in \mathcal{X}$ :

$$f(x) = dF(x)/dx \\ = F'(x)$$

Cumulative Distribution Function (CDF)

$$F(x) = P(X \leq x) \\ = \int_{-\infty}^x f(t)dt$$

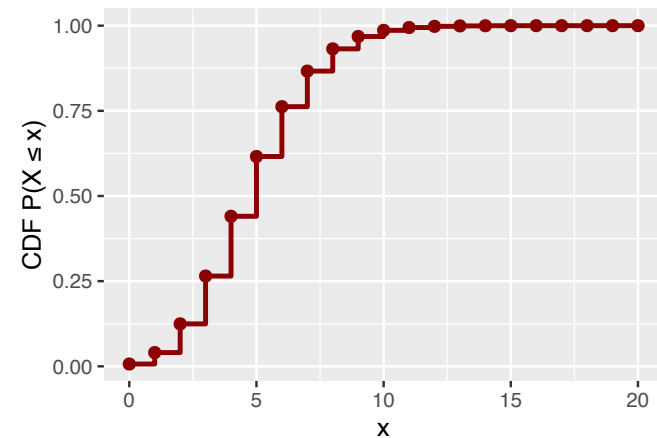
Quantile Function or inverse CDF

$$F^{-1}(p) = \inf\{x : \\ F(x) \geq p\} \\ \text{for } p \in [0, 1]$$

These definitions correspond to the R functions: `dbinom`, `pbinom`, `qbinom`.

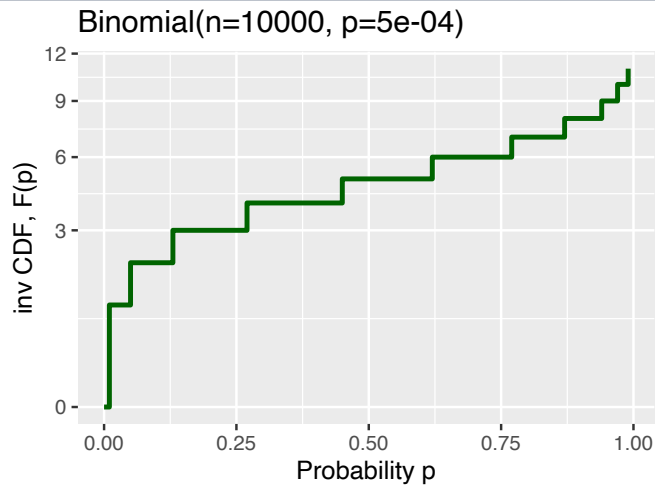
## Cumulative Distribution function pbinom

Binomial(n=10000, p=5e-04)



$F(x) = P(X \leq x)$  cumulative probability up to  $x$

## Quantile function qbinom



$F^{-1}(p)$  gives the value  $x$  such that  $P(X \leq x) \geq p$

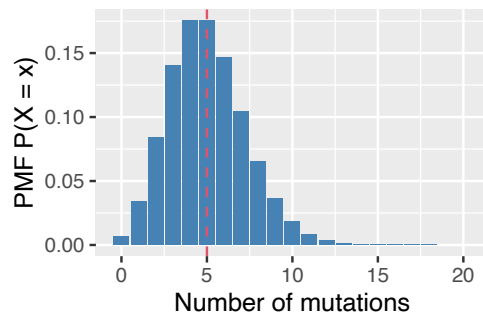
## Expectation (the 1st moment)

- What if we have  $n = 10^4$  nucleotides?
- We still assume that each nucleotide can have mutation with  $p = 5 \times 10^{-4}$ .

$$X \sim \text{Binomial}(n, p)$$

Overall, what is the expected number of mutations?

$$\mathbb{E}[X] = np = 10^4 \cdot 5 \times 10^{-4} = 5$$



## Two types of random variables

- A **discrete** RV has a countable number of possible values
  - e.g. coin flip, throwing dice, genotype at a particular locus
- A **continuous** RV takes on values in an interval of numbers
  - e.g. blood glucose level, height of individuals

## Formal definitions: Moments of a distribution

### Expectation ( $\mu$ )

Measures the average:

For discrete,  
 $\mathbb{E}[X] = \sum_{x \in \mathcal{X}} x \cdot f(x)$

For continuous,

$$\mathbb{E}[X] = \int_{x \in \mathcal{X}} x \cdot f(x) dx$$

### Variance ( $\sigma^2$ )

Measures the spread:

$$\mathbb{V}[X] = \mathbb{E}[(X - \mu)^2]$$

Equivalently:

$$\mathbb{V}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Standard deviation:

$$\sigma = \sqrt{\mathbb{V}[X]}$$

### Skewness

Measures asymmetry of the distribution:

$$\gamma_1 = \mathbb{E} \left[ \left( \frac{X - \mu}{\sigma} \right)^3 \right]$$

Equivalently:

$$\gamma_1 = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3}$$

## Moments of Binomial distribution

For  $X \sim \text{Binomial}(n, p) \equiv \text{Binomial}(n = 10^4, p = 5 \cdot 10^{-4})$ :

Expectation

$$\mathbb{E}[X] = np$$

Variance

$$\mathbb{V}[X] = np(1 - p)$$

Standard deviation:

$$\sigma = \sqrt{np(1 - p)}$$

Skewness

$$\gamma_1 =$$

$$(1 - 2p) / (\sqrt{np(1 - p)})$$

```
mu <- n * p
sig2 <- n * p * (1 - p)
gam <- (1 - 2 * p) / sqrt(n * p * (1 - p))

c(mu, sig2, gam)
```

```
## [1] 5.0000000 4.9975000 0.4468781
```

## Poisson distribution for the number of mutations

For Poisson RV,  $X \sim \text{Poisson}(\lambda)$ , we have the distribution function:

$$f(x) \rightarrow \frac{\lambda^x e^{-\lambda}}{x!}, \quad x = 0, 1, 2, \dots$$

Moments:

$$\mathbb{E}[X] = \lambda, \quad \mathbb{V}[X] = \lambda$$

## What if the genome is very long, mutation is very rare

For  $X \sim \text{Binomial}(n, p)$ , probability function:

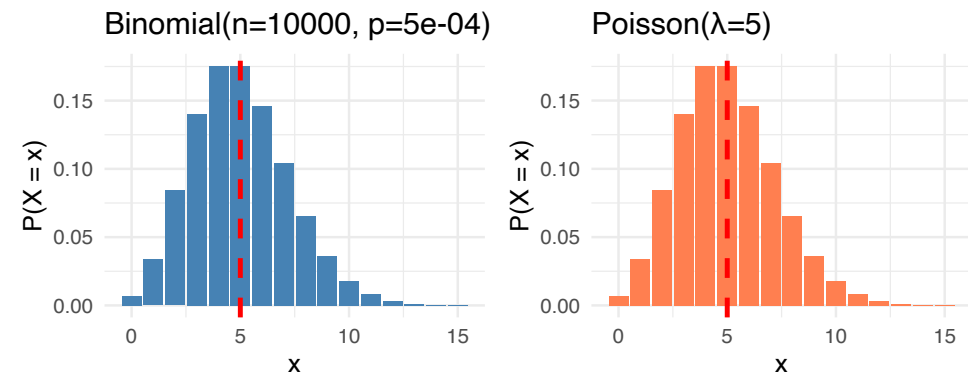
$$f(x) = \binom{n}{x} p^x (1 - p)^{n-x}$$

- Very long genome:  $n \rightarrow \infty$
- Very rare mutation:  $p \rightarrow 0$
- But finite number of mutations:  $np = \lambda < \infty$

## Compare Binomial and Poisson

With the same parameters:

$$n = 10^4, p = 5 \times 10^{-4}, \lambda = np = 5$$

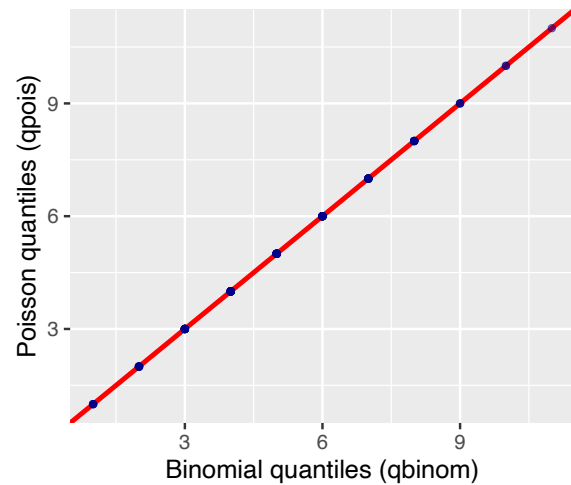


Binomial quantile function

Poisson quantile function



## QQ-plot: Binomial vs Poisson



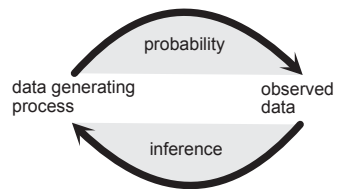
## An example: $X \sim \text{Poisson}(\lambda = 5)$

Let's see what happens when we take sample means from  $X \sim \text{Poisson}(5)$ .

```
lambda <- 5  
xv <- rpois(10, lambda)  
print(xv)
```

```
## [1] 5 3 6 3 4 7 10 3 5 2
```

## Statistical Inference is to figure out the unknown of a data generating process



We knew (or assumed)

$$X \sim \text{Poisson}(\lambda)$$

and had some observed data set

$D = \{$   
5, 3, 6, 3, 4, 7, 10, 3, 5, 2  
 $\dots\}$

► Can we figure out the unknown  $\lambda$ ?

## How can we estimate the unknown parameters?

- Assume  $x_i \in D$  values are observation of independent and identically distributed (i.i.d.)  $X$  RV.

## Can we estimate mean parameter by sample mean?

### (Weak) Law of Large Numbers

Let  $X_1, X_2, \dots, X_n$  be i.i.d. with the mean  $\lambda$

The sample mean converges to the true mean  $\lambda$ :

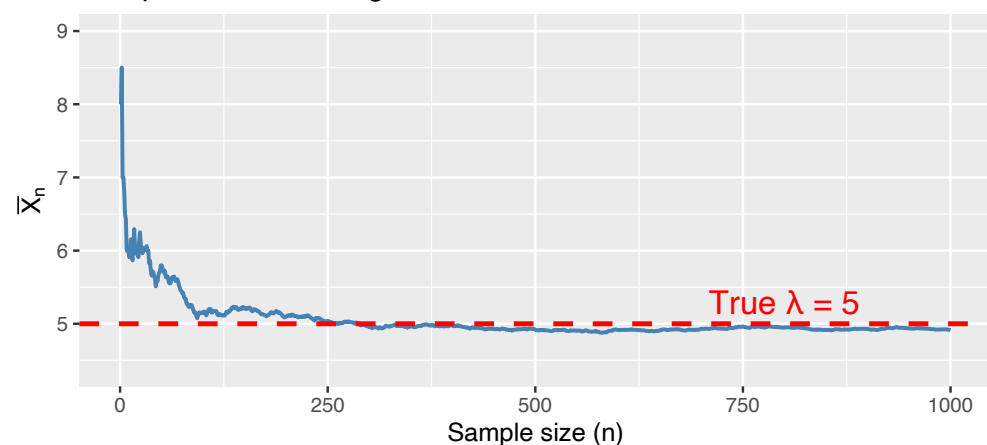
$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{P} \lambda$$

as  $n \rightarrow \infty$ .

- ▶ With more observations,  $n \rightarrow \infty$ , the sample mean gets closer to the true mean
- ▶  $\xrightarrow{P}$ : This is **convergence in probability**, meaning  $P(|\bar{X}_n - \lambda| > \epsilon) \rightarrow 0$  for any  $\epsilon > 0$
- ▶ Using sample mean to estimate mean is okay with high probability.

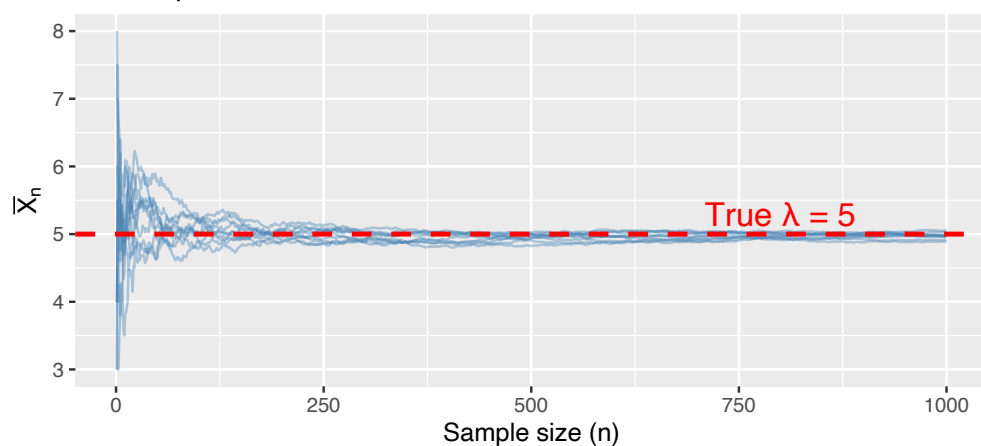
## Example: Estimating $\lambda$ from Poisson(5)

Sample mean converges to true mean



## Multiple realizations of the LLN

10 independent realizations



## Central Limit Theorem

### Central Limit Theorem

Let  $X_1, X_2, \dots, X_n$  be i.i.d. with mean  $\mathbb{E}[X_1] = \mu$  and variance  $\mathbb{V}[X_1] = \sigma^2$ .

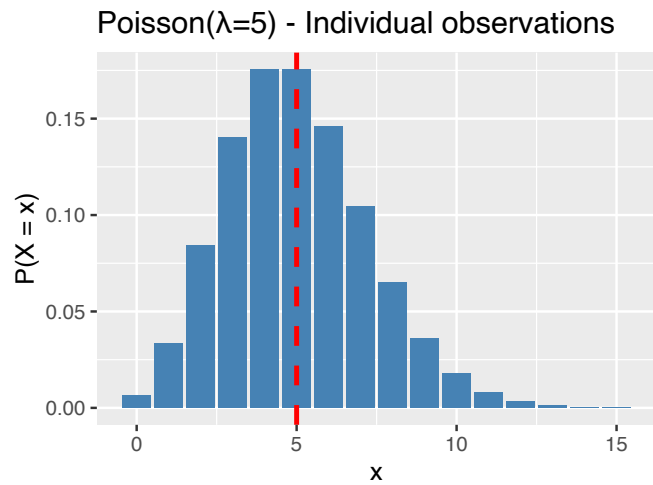
The sample mean  $\bar{X}_n = n^{-1} \sum_{i=1}^n X_i$  converges in distribution to a normal distribution:

$$\bar{X}_n \xrightarrow{d} \mathcal{N}(\mu, \sigma^2/n)$$

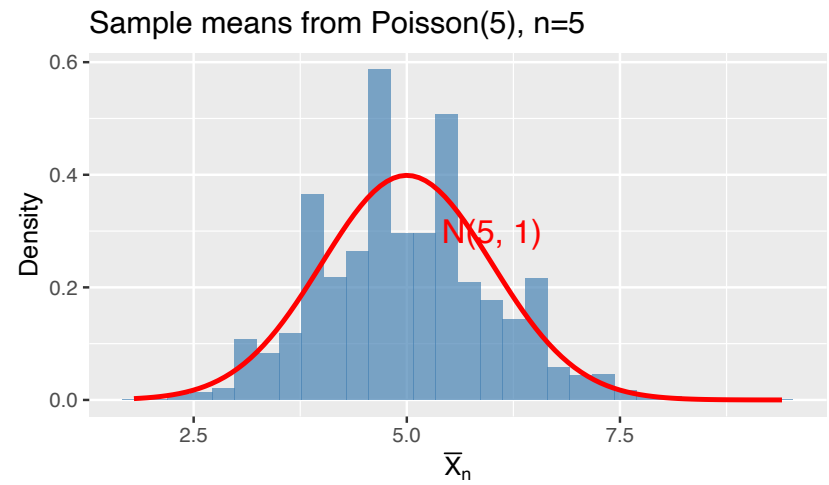
as  $n \rightarrow \infty$ .

- ▶ For any distribution, CLT holds!
- ▶ The distribution of sample mean  $\bar{X}_n$  becomes approximately normal for large  $n$
- ▶ The variance of  $\bar{X}_n$  decreases as  $n$  increases (how fast?)

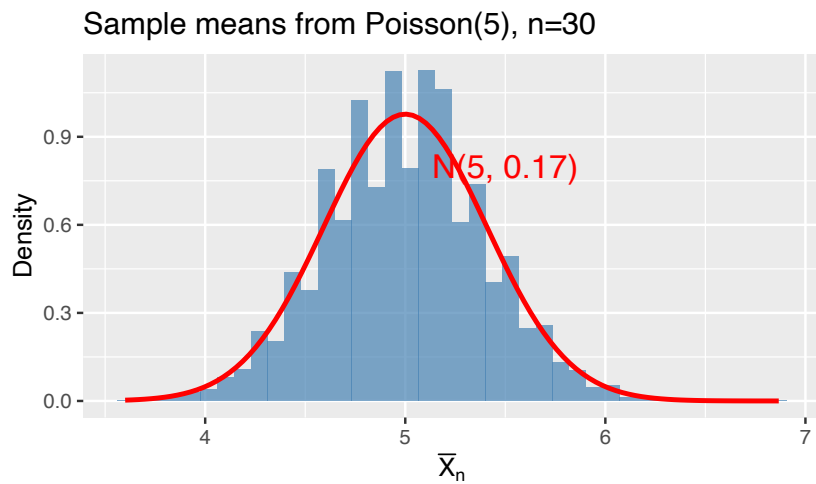
## The sample Poisson example



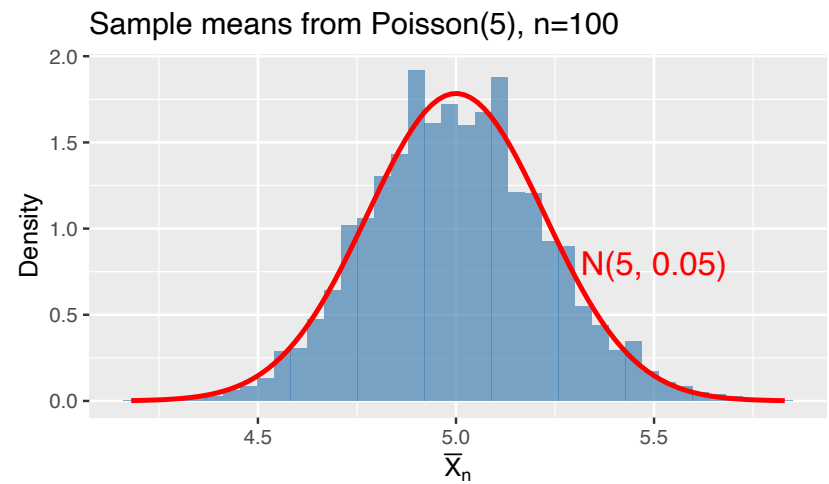
## Sample means with data size $n=5$



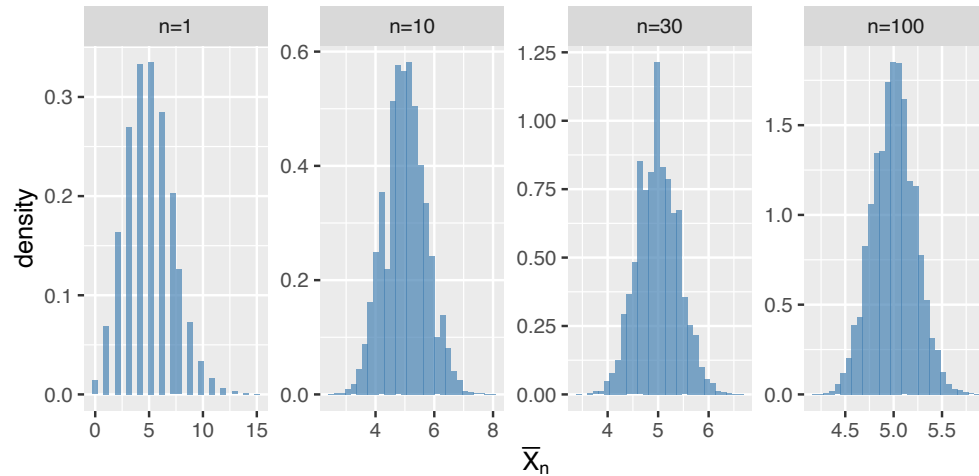
## Sample means with $n=30$



## Sample means with $n=100$



## Different (data) sample sizes (10k simulations)



## Some clarifications

Given  $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ , i.i.d.,  $X_i \sim \mathcal{N}(\mu, \sigma^2)$

can you distinguish the following?

$$\mathbb{E}[X_1]$$

vs.

$$\mathbb{E}[\bar{X}_n]$$

$$\mathbb{V}[X_1]$$

vs.

$$\mathbb{V}[\bar{X}_n]$$

## Some clarifications

Given  $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ , i.i.d.,  $X_i \sim \mathcal{N}(\mu, \sigma^2)$

can you distinguish the following?

$$\mathbb{E}[X_1] = \mu$$

vs.

$$\mathbb{E}[\bar{X}_n] = \mu$$

$$\mathbb{V}[X_1] = \sigma^2$$

vs.

$$\mathbb{V}[\bar{X}_n] = \sigma^2/n$$

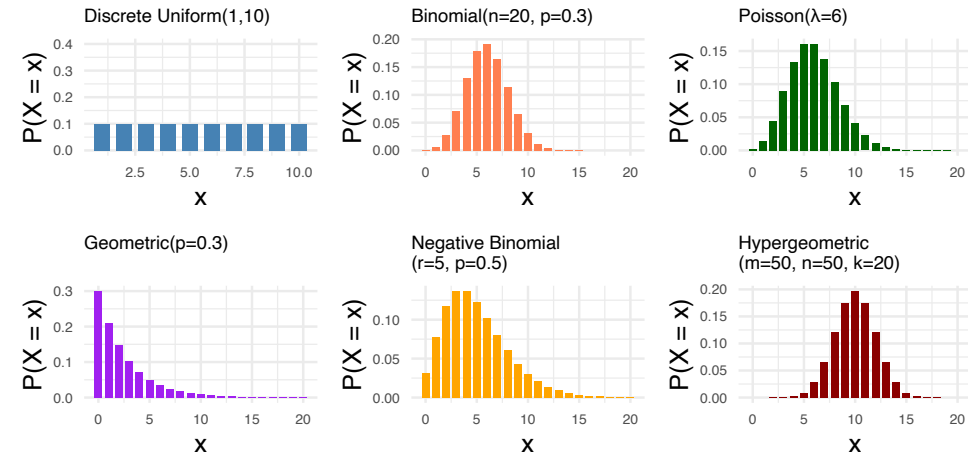
## What CLT suggests in real world applications...

- If your variable of interest is something like sample mean?
- If you have enough sample size?

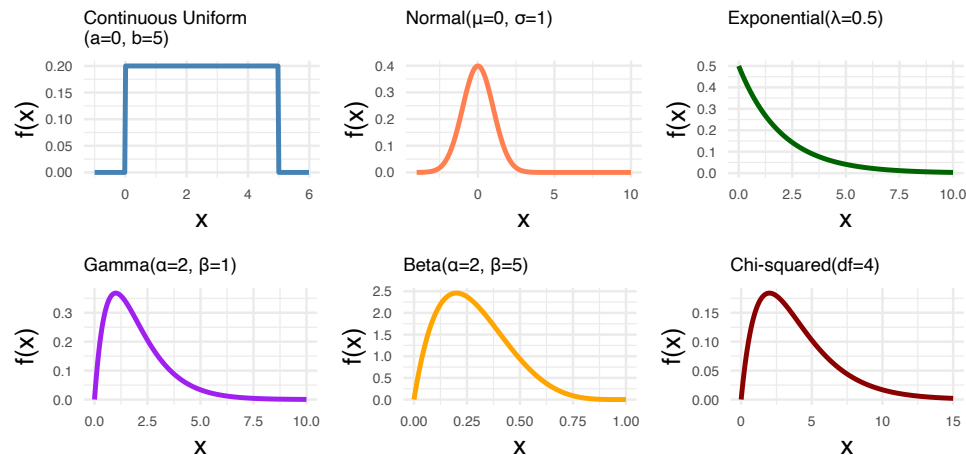
## Can you suggest probability distributions for these?

- ▶ number of chromosomes
- ▶ survival time
- ▶ mRNA expression level

## Common Discrete Distributions



## Common Continuous Distributions



How do we know **which distribution** works well with our own data?

Should we use just one type of distribution or many?

## Summary

---

- ▶ Random variables are variables that have a probability distribution
- ▶ Any statistic of sampled data is a RV, and hence has an associated probability distribution
- ▶ The CLT gives us the sampling distribution of the mean of any RV (regardless of its distribution)
- ▶ We can use statistical inference to estimate population parameters from a sample

## Pop quiz: What are they?

---

```
## 1.  
rnorm(...)
```

```
## 2.  
dnorm(...)
```

```
## 3.  
pnorm(...)
```

```
## 4.  
qnorm(...)
```

```
import numpy as np  
import scipy.stats as stats
```

```
## 1.  
np.random.normal(...)
```

```
## 2.  
stats.norm.pdf(...)
```

```
## 3.  
stats.norm.cdf(...)
```

```
## 4.  
stats.norm.ppf(...)
```

## Today's lecture

---

Statistics Refresher

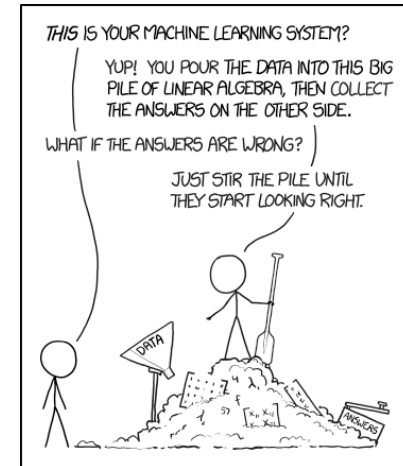
Matrix Algebra Primer

Matrix Algebra in Statistics

## Why do we use matrix?

Two main reasons:

1. **Simplify mathematical notation:** Describe many linear models concisely in one line
  2. **Efficient computation:** many programming languages (Python, R, C/C++, Fortran) implement matrix operations efficiently (a library built on decades of optimization)
- ▶ Matrix (linear) algebra is a foundational tool to represent whole space of vectors (many variables).
  - ▶ Our long-term goal: We want to think of a multivariate linear regression in terms of “column space.”



Source: xkcd #1838: Machine Learning

## Example: A system of linear equations

Solve for  $a$ ,  $b$ ,  $c$ :

$$\begin{aligned}a + b + c &= 6 \\ 3a - 2b + c &= 2 \\ 2a + b - c &= 1\end{aligned}$$

Matrix form:

$$\begin{bmatrix} 1 & 1 & 1 \\ 3 & -2 & 1 \\ 2 & 1 & -1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \\ 1 \end{bmatrix}$$

More compact:  $\mathbf{Ax} = \mathbf{b}$

Source: Matrix Notations

## Matrix Notation: Scalars, Vectors, Matrices

### Scalar

Single number

$$a = 5$$

$$\lambda = 3.14$$

### Vector

Ordered collection

(bold)

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$n \times 1$  matrix

### Matrix

Rectangular array (bold)

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} \\ x_{2,1} & x_{2,2} \\ \vdots & \vdots \\ x_{n,1} & x_{n,2} \end{bmatrix}$$

$n \times p$  matrix

Source: Matrix Notations

## Notation conventions

- ▶ **(Bold) uppercase** for matrices: **X**, **A**
- ▶ **Bold lowercase** for vectors: **y**, **x**
- ▶ Regular lowercase for scalars:  $a$ ,  $x_{i,j}$
- ▶  $x_{i,j}$  denotes element in row  $i$ , column  $j$  of matrix **X**
- ▶ Vectors are typically  $n \times 1$  (column vectors) by convention (statistics)
- ▶ Vectors are usually  $1 \times n$  (row vectors) in many AI/deep learning libraries

Source: Matrix Notations

## Example in R: Creating matrices

```
# Create a vector
y <- c(1, 2, 3, 4, 5)
y

## [1] 1 2 3 4 5

# Create a matrix by binding columns
X <- cbind(c(1, 1, 1), c(1, 2, 3))
X

##      [,1] [,2]
## [1,]    1    1
## [2,]    1    2
## [3,]    1    3
```

## Creating matrices in R (continued)

```
# Using matrix() function (fills column by column by
# default)
A <- matrix(c(1, 3, 2, 1, -2, 1, 1, 1, -1), nrow = 3, ncol = 3)
A

##      [,1] [,2] [,3]
## [1,]    1    1    1
## [2,]    3   -2    1
## [3,]    2    1   -1
```

## Creating matrices in R (continued)

```
# Fill row by row instead
B <- matrix(c(1, 1, 1, 3, -2, 1, 2, 1, -1), nrow = 3, ncol = 3,
            byrow = TRUE)
B

##      [,1] [,2] [,3]
## [1,]    1    1    1
## [2,]    3   -2    1
## [3,]    2    1   -1
```



## Pop quiz

Is it row-major or column major?

```
matrix(...)
```

## Scalar Multiplication

Multiply each element by the scalar:

$$c\mathbf{X} = c \begin{bmatrix} x_{1,1} & x_{1,2} \\ x_{2,1} & x_{2,2} \end{bmatrix}$$
$$= \begin{bmatrix} c \cdot x_{1,1} & c \cdot x_{1,2} \\ c \cdot x_{2,1} & c \cdot x_{2,2} \end{bmatrix}$$

```
X <- matrix(c(1, 2, 3, 4), 2, 2)
X
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
# Scalar multiplication
3 * X
##      [,1] [,2]
## [1,]    3    9
## [2,]    6   12
```

## Pop quiz

Is it row-major or column major?

```
matrix(..., byrow = TRUE)
```

## Transpose

Converts rows into columns and columns into rows:  $\mathbf{X}^T$

If  $\mathbf{X}$  is  $n \times p$ , then  $\mathbf{X}^T$  is  $p \times n$

$$\mathbf{X} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \\ 3 & 5 \end{bmatrix}_{3 \times 2}$$

$$\mathbf{X}^T = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \end{bmatrix}_{2 \times 3}$$

```
X <- matrix(c(1, 2, 3, 3, 4, 5), 3, 2)
X
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
## [3,]    3    5
# Transpose
t(X)
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    3    4    5
```

## Matrix Multiplication

**Check:** Number of columns in first matrix = number of rows in second matrix.

If **A** is  $n \times p$  and **B** is  $p \times m$ , then **AB** is  $n \times m$ .

$$(\mathbf{AB})_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}$$

Row  $i$  of **A** times column  $j$  of **B**

```
A <- matrix(c(1, 2, 3, 4), 2, 2)
B <- matrix(c(5, 6, 7, 8), 2, 2)

# Matrix multiplication
A %*% B

##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

Source: Matrix Operations

## Matrix Multiplication

Note:

```
# Matrix multiplication
A %*% B

##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

Source: Matrix Operations

This is different from:

```
# Element-wise multiplication
A * B

##      [,1] [,2]
## [1,]    5   21
## [2,]   12   32
```

## Matrix Multiplication Example

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 + 3 \cdot 6 & 1 \cdot 7 + 3 \cdot 8 \\ 2 \cdot 5 + 4 \cdot 6 & 2 \cdot 7 + 4 \cdot 8 \end{bmatrix}$$
$$= \begin{bmatrix} 23 & 31 \\ 34 & 46 \end{bmatrix}$$

```
matrix(c(1, 2, 3, 4), 2, 2) %*% matrix(c(5, 6, 7, 8), 2, 2)

##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

Source: Matrix Operations

## Identity Matrix

$n \times n$  square matrix with ones on diagonal, zeros elsewhere:  $\mathbf{I}_n$

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Property:  $\mathbf{XI} = \mathbf{IX} = \mathbf{X}$  (like multiplying by 1)

```
I <- diag(3)
I

##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
```

Source: Matrix Operations

## Matrix Inverse

For square matrix  $\mathbf{A}$ , the inverse  $\mathbf{A}^{-1}$  satisfies:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$$

Not all matrices have inverses!

```
A <- matrix(c(1, 3, 2, 1, -2, 1, 1, 1, -1), 3, 3, byrow = TRUE)
A_inv <- solve(A)
A_inv
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.07692308 0.3846154 0.53846154
## [2,] 0.15384615 -0.2307692 0.07692308
## [3,] 0.23076923 0.1538462 -0.38461538
```

## Solving linear equations with matrix inverse

Recall:  $\mathbf{Ax} = \mathbf{b}$

$$\begin{aligned}\mathbf{A}^{-1}\mathbf{Ax} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{Ix} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{x} &= \mathbf{A}^{-1}\mathbf{b}\end{aligned}$$

```
## solve linear equations
x <- solve(A, b)
round(x, 2)
## [1] 1.77 0.54 1.31

## another way (less efficient)
x <- solve(A) %*% b
round(t(x), 2)
##           [,1] [,2] [,3]
## [1,] 1.77 0.54 1.31
```

## Verify: Should give identity matrix

For square matrix  $\mathbf{A}$ , the inverse  $\mathbf{A}^{-1}$  satisfies:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$$

```
A %*% A_inv
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.000000e+00 0.000000e+00 0.000000e+00
## [2,] 0.000000e+00 1.000000e+00 -1.110223e-16
## [3,] 1.110223e-16 -2.775558e-17 1.000000e+00
```

## Summary

- A system of linear equations:

$$\mathbf{Ax} = \mathbf{b}$$

- Matrix operations: **multiplication, transpose, inverse**
- Recipe: **The Matrix Cookbook**

## Today's lecture

---

### Statistics Refresher

### Matrix Algebra Primer

### Matrix Algebra in Statistics

## Sample mean using matrix operations

---

The sample mean:  $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$

Using matrices: Define  $\mathbf{1}_n = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$  (vector of ones)

$$\bar{y} = \frac{1}{n} \mathbf{1}_n^\top \mathbf{y}$$

```
y <- c(3, 5, 7, 2, 8)
n <- length(y)
ones <- rep(1, n)
```

```
## Sample mean using matrix multip
mean.y <- (1/n) * t(ones) %*% y
mean.y
```

```
##      [,1]
## [1,]    5
```

```
## check
all(mean(y) == mean.y)
```

```
## [1] TRUE
```

## Calculating moments with matrices

---

Goal: Express mean and variance using matrix operations

Given data vector  $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$

## Centering data with matrices

---

Centered data:  $y_i - \bar{y}$  for all observations

Matrix form:  $\mathbf{y} - \bar{y} \mathbf{1}_n$

```
y <- c(3, 5, 7, 2, 8)
mean.y <- mean(y)
n <- length(y)
```

```
## Centering
ones.n <- rep(1, n)
y.cent <- y - mean.y * ones.n
y.cent
```

```
## [1] -2  0  2 -3  3
```

```
mean(y.cent)
```

```
## [1] 0
```

## Sample variance using matrix operations

Sample variance:  $s^2 = (n - 1)^{-1} \sum_{i=1}^n (y_i - \bar{y})^2$

Letting (centered data)

$$\tilde{\mathbf{y}} = \mathbf{y} - \bar{y}\mathbf{1}_n$$

$$s^2 = (n - 1)^{-1} \tilde{\mathbf{y}}^\top \tilde{\mathbf{y}}$$

## Sample variance using matrix operations

```
y <- c(3, 5, 7, 2, 8)
n <- length(y)
y.tilde <- y - mean(y)
```

```
# using matrix multiplication
var.y <- t(y.tilde) %*% y.tilde
var.y <- var.y/(n - 1)
```

```
# Compare with built-in var
all(var.y == var(y))
```

```
## [1] TRUE
```

## Covariance matrix using matrix operations

For two variables  $X$  and  $Y$ , the covariance matrix is:

$$\Sigma = \begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(X, Y) & \text{Var}(Y) \end{bmatrix}$$

With centered data matrix  $\tilde{\mathbf{X}}$  ( $n \times 2$ ):

$$\Sigma = \frac{1}{n - 1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$$

## Computing 2x2 covariance matrix in R

```
# Two variables
x1 <- c(3, 5, 7, 2, 8)
x2 <- c(4, 6, 5, 3, 7)
n <- length(x1)
```

```
# Create data matrix
X <- cbind(x1, x2)
X
```

```
##      x1 x2
## [1,]  3  4
## [2,]  5  6
## [3,]  7  5
## [4,]  2  3
## [5,]  8  7
```

```
# Center the data
X.tilde <- scale(X, scale = FALSE)
X.tilde
```

```
##      x1 x2
## [1,] -2 -1
## [2,]  0  1
## [3,]  2  0
## [4,] -3 -2
## [5,]  3  2
## attr(,"scaled:center")
##      x1 x2
##      5  5
```

## Computing 2x2 covariance matrix in R (continued)

---

```
# Covariance matrix
Sigma <- t(X.tilde) %*% X.tilde
Sigma <- Sigma/(n - 1)
Sigma

##      x1  x2
## x1 6.5 3.5
## x2 3.5 2.5
```

```
# Compare with built-in cov
cov(X)

##      x1  x2
## x1 6.5 3.5
## x2 3.5 2.5
```

## Summary: Why use matrix notation?

---

1. **Concise representation:** Express complex operations compactly
2. **Generalization:** Easily extend to multivariate data (multiple variables)
3. **Computational efficiency:** Matrix libraries are highly optimized